Applying the Memory Measurement Model (M^3): A Tutorial Using the bmm R Package

Gidon T. Frischkorn[1] and Chenyu Li[2,3]

[1]Department of Psychology, University of Zurich

[2]Department of Psychology, University of Chicago

[3]Institution for Mind and Biology, University of Chicago

Author Note

Gidon T. Frischkorn https://orcid.org/0000-0002-5055-9764

Chenyu Li https://orcid.org/0000-0001-8815-6189

This study was not preregistered. Data, analysis code, and companion R scripts for all four tutorials are available on OSF and GitHub. The authors declare no conflicts of interest. This research was supported by a grant from the Swiss National Science Foundation (SNSF) awarded to Gidon T. Frischkorn (Grant No. 208828). We thank Isabel Courage for valuable feedback during the development of the M3 implementation in the bmm R package.

Author Contributions: Gidon T. Frischkorn: Conceptualization, Methodology, Software, Formal Analysis, Visualization, Writing – Original Draft. Chenyu Li: Methodology, Software, Data Curation, Writing – Review & Editing.

Correspondence concerning this article should be addressed to Gidon T. Frischkorn, Department of Psychology, University of Zurich, Email: gidon.frischkorn@psychologie.uzh.ch

Abstract

The memory measurement model (M^3) decomposes categorical response data from memory tasks into latent activation parameters, separating item memory from context-specific binding via competitive retrieval. This tutorial introduces the M^3 framework and demonstrates how to apply it to empirical data using the bmm R package, which provides a standardized interface for fitting Bayesian hierarchical measurement models. Four tutorial examples build progressively: In Tutorials 1 and 2, we fit M^3 to two common working memory paradigms, simple span and complex span, demonstrating how different choice rules can be applied and how distractor categories extend the basic model. In Tutorial 3, we use the complex span data to illustrate how to build a custom specification based on different theoretical assumptions. Tutorial 4 provides a simulation-based workflow for validating custom specifications through parameter recovery. Complete R scripts accompany each tutorial. After completing the tutorials, readers will be able to apply standard and custom M^3 to their own data and validate new model specifications through simulation.

Keywords: working memory, measurement model, multinomial model, Bayesian estimation, categorical data

Applying the Memory Measurement Model (M^3): A Tutorial Using the bmm R Package

Observed performance in memory and decision tasks is often summarized by overall accuracy. Yet, for many tasks accuracy collapses across qualitatively distinct response categories and error types that may arise from different latent processes. In particular, different error types are diagnostically informative because different cognitive processes imply different conditional response probabilities over observable outcomes. Multinomial processing tree (MPT) models formalize this logic by treating categorical responses - including distinct error classes - as consequences of latent states such as successful retrieval, guessing, or source confusion (Batchelder & Riefer, 1999; Erdfelder et al., 2009). Similarly, process-dissociation approaches demonstrate that patterns of correct and incorrect responses across task constraints can separate the contributions of multiple processes within a single task (Jacoby, 1991; Yonelinas & Jacoby, 2012). In sum, the full multinomial response distribution - not accuracy alone - carries more information about latent cognitive mechanisms than a single accuracy score.

Working memory research frequently relies on tasks that inherently yield response frequencies across discrete outcome categories, such as correct responses, feature confusions, location errors, or intrusions (Oberauer et al., 2018). For discrete feature spaces (e.g., words, numbers, letters), these data are naturally multinomial: each retrieval produces one of several mutually exclusive response types. Thus, theoretical claims can often be evaluated by how experimental manipulations and individual differences redistribute probability mass across these categories.

The recently proposed memory measurement model M^3 (Oberauer & Lewandowsky, 2019) framework provides one such decomposition. Its core assumptions are that (a) multiple

activation sources contribute to each response category and that (b) selection among categories follows a competitive retrieval rule (Luce, 1959). Although the M^3 framework was introduced for working memory tasks, the general architecture is domain-general: whenever a task yields categorical responses that can be traced to distinct latent activation sources driving competitive selection, the framework applies.[1]

Because the M^3 framework can be applied to different paradigms — including simple span (Loaiza & Souza, 2024; e.g., Oberauer, 2019), complex span (Li et al., 2026; Schubert et al., 2023), and memory updating tasks (Li, Frischkorn, & Oberauer, 2025; Li, Frischkorn, Dames, et al., 2025) — and can incorporate varied processes such as distractor filtering or removal of outdated information, the activation formulas that drive response selection typically vary across tasks and response categories. Beyond these existing applications, researchers can develop their own model specifications tailored to their paradigm and theoretical questions. Each new application, however, has so far required translating the model into custom JAGS, Stan, or `brms` code using advanced non-linear formula syntax, making implementations error-prone and hard to reuse. The `bmm` R package (Popov et al., 2025) addresses this limitation by providing a standardized, low-code workflow for specifying, fitting, and evaluating Bayesian hierarchical M^3, shifting the burden of routine implementation details away from users and toward theory-driven model development and interpretation. In this tutorial, we illustrate this approach with

[1] Readers familiar with multinomial processing tree (MPT) models will note similarities: both frameworks decompose categorical response distributions into latent process parameters. However, whereas MPT models use discrete processing trees with binary branching probabilities (Batchelder & Riefer, 1999; e.g., Heck et al., 2018 for Bayesian hierarchical implementations), M^3 uses continuous activation values with competitive selection, making it particularly suited for theories emphasizing graded memory strength (for a detailed comparison, see Oberauer & Lewandowsky, 2019).

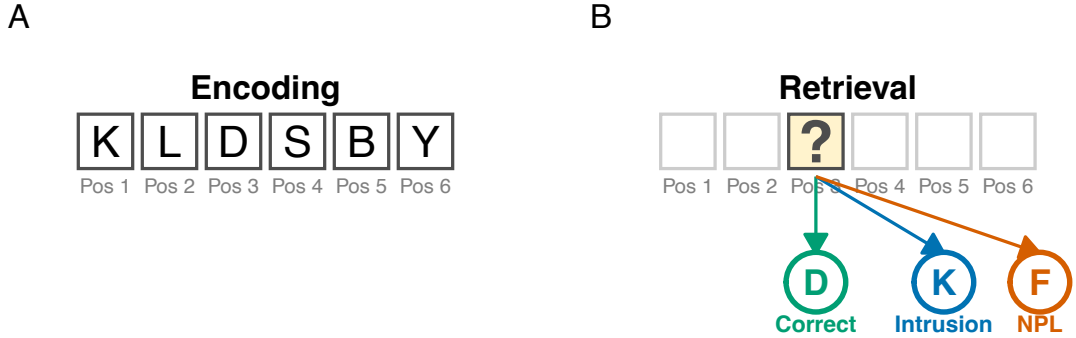
four worked examples covering the standard simple span (ss) and complex span (cs) tasks, a custom extension, and parameter recovery as model validation.

The M^3 Framework

Consider a cued recall task in which a participant studies a sequence of items (e.g., letters at serial positions or icons at spatial locations) and later sees a retrieval cue and selects a response from a set of alternatives. Figure 1 illustrates this task structure for a simple span paradigm. As shown in the figure, each response falls into one of three categories: *correct* — the item associated with the probed position, *other-list intrusion* — a studied item from a different position, or *not-presented lure* (NPL) — an item not presented during the trial. The M^3 framework explains each category's selection frequency by specifying activation formulas: different activation sources feed into each category, and response probabilities arise as the result of applying choice rules that implement competitive selection among all options.

Figure 1

Simple span cued recall task. During encoding (A), participants study a sequence of letters at serial positions. At retrieval (B), a position is probed and the participant selects a response. Each response falls into one of three categories: correct (the target at the probed position), intrusion (a studied item from another position), or not-presented lure (NPL).



Activation Formulas and Model Parameters

Each response category's activation formula specifies which latent sources contribute to its selection probability. Because the M^3 framework is not committed to any particular memory theory, different assumptions about the processes limiting memory performance can be accommodated through different activation formulas.

Each response category usually receives activation from one or more latent sources. For the cued recall task introduced above, the theoretically assumed activation sources are:

- b: **baseline activation**, the tendency to select any response option in the absence of memory-based evidence. This could reflect guessing, response bias, or other non-mnemonic factors that contribute to selection probabilities across all categories.
- a: **item activation**, or general memory strength for studied items. An item that was encountered during study receives this activation regardless of its position or context.
- c: **context binding**, the strength of the association between an item and its specific study context (e.g., serial position, paired associate).

These activation sources or additional mechanisms are specified as parameters that can vary and are estimated from data. Using these parameters, we can specify how they are combined to produce total activations for each response category. The simple span model proposed by Oberauer and Lewandowsky (2019) assumes that these activation sources combine additively, with the correct target receiving activation from all three sources, other studied items receiving item activation but lacking context binding, and lures receiving only baseline activation:

$$\begin{aligned} A_{\text{correct}} &= b + a + c \\ A_{\text{other}} &= b + a \\ A_{\text{lures}} &= b \end{aligned}$$

Another perspective on the interplay of these processes could also assume that context activation boosts item activation, such that the correct target receives a multiplicative boost from context binding, while other studied items receive only item activation:

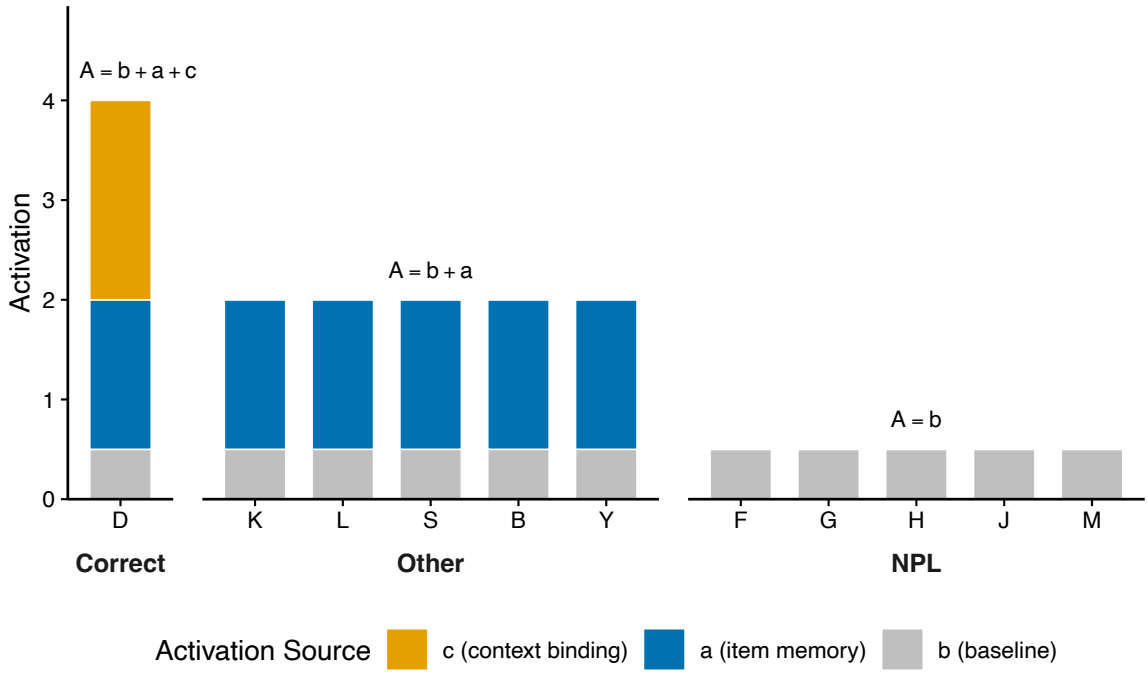
$$\begin{aligned} A_{correct} &= b + a \cdot (1 + c) \\ A_{other} &= b + a \\ A_{lures} &= b \end{aligned}$$

These two examples illustrate how distinct theoretical perspectives can be encoded through the activation formulas.[2] Figure 2 visualizes the additive formulas from the simple span example: each response option receives activation from a subset of the three sources, with the correct target accumulating the most activation. A choice rule then converts these activations into response probabilities (see next section). The M^3 framework includes pre-specified formulas for simple span and complex span versions (Oberauer & Lewandowsky, 2019), and supports custom formulas for novel task designs.

[2] Although the M^3 framework was conceived as a measurement tool, it can thus be used to implement and test specific theoretical assumptions. For example, similarity between spatial positions could be implemented by other list items getting a partial boost from context binding dependent on their distance to the currently cued position. Thereby the M^3 framework sits at the intersection of measurement and process models, it can account for the interplay of different processes, but does not provide any implementation of the activation sources and why they might differ in strength.

Figure 2

Activation decomposition for each response option in the simple span example from Figure 1. Bars show the total activation for each option, decomposed into baseline (b, grey), item memory (a, blue), and context binding (c, orange). The correct target (D) receives all three sources ($b + a + c$), other studied items receive item memory and baseline ($b + a$), and not-presented lures receive only baseline (b). A choice rule converts these activations into response probabilities.



Competitive Selection and Choice Rules

To link activations to observed responses, each category's activation must be converted into a response probability. The M^3 framework assumes competitive selection, that is a category's probability depends on its activation relative to all competitors. For implementing any choice rules that implement competitive selection the number of all competitors, that is options per category, needs to be known. Therefore, tasks designed for M^3 should use a recognition format in which the experimenter controls the available response options instead of free-recall where participants generate their response set internally.

Given these option counts, M^3 provides two choice rules that differ in how activations are transformed before normalization. The *simple* (or linear) choice rule (Luce, 1959) normalizes activations directly:

$$P(k) = \frac{n_k \cdot \mathrm{A}_k}{\sum_{j=1}^{K} n_j \cdot \mathrm{A}_j}$$

where A_k is the total activation of an item for category k and n_k is the number of response options in that category. This rule requires all activations to be positive. To identify the scaling of activations, the baseline b is fixed at a small positive value (0.1 by default).

The *softmax* (or exponential) choice rule applies an exponential transformation before normalization:

$$P(k) = \frac{n_k \cdot \exp(\mathrm{A}_k)}{\sum_{j=1}^{K} n_j \cdot \exp(\mathrm{A}_j)}$$

Because $\exp(x) > 0$ for all x, softmax guarantees valid probabilities regardless of activation values. Again, to identify the scaling of activations, the baseline is fixed at $b = 0$ (since $\exp(0) = 1$, lures receive a positive baseline contribution automatically).

The softmax rule is formally equivalent to an n-alternative forced-choice signal detection model with Gumbel-distributed noise (Thurstone, 1927; Yellott, 1977). Under this interpretation, each category's activation represents a deterministic signal, independent Gumbel noise is added to each alternative, and the observer selects the category with the highest noisy value. This equivalence gives activation parameters a principled interpretation as discriminability measures. Recent empirical comparisons have favored the softmax rule over the simple rule for working memory data (Li et al., 2026; Oberauer, 2023) although the simple choice rule was endorsed

initially (Oberauer & Lewandowsky, 2019). Tutorial 1 compares both rules; Tutorials 2–4 use the softmax rule as the default.

Model Versions

Oberauer and Lewandowsky (2019) proposed two concrete M^3 specifications for canonical memory paradigms, which differ in their response categories and estimated parameters. Beyond these, custom specifications can be defined for novel task designs. We briefly introduce each; the tutorials provide details.

The **simple span** (`ss`) version implements M^3 for cued recall tasks separating three response categories (correct response, other, lures) with two free parameters (a, c). It implements the additive activation formula proposed by Oberauer and Lewandowsky (2019).

The **complex span** (`cs`) version extends the simple span to tasks that interleave distractor processing with memorization. It requires that memoranda and distractors are drawn from the same material so that distractors can plausibly intrude during retrieval (Li et al., 2026; for examples, see Oberauer & Lewandowsky, 2019). The `cs` version separates five response categories (correct, paired distractor, other memorandum, other distractor, and NPL) with three free parameters (a, c, f). The additional parameter f captures distractor filtering efficiency on a scale from 0 to 1: $f = 0$ indicates perfect filtering (distractors receive only baseline activation), while $f = 1$ indicates no filtering (distractors are activated as strongly as memoranda).

The **custom** version allows user-defined response categories, activation formulas, and parameters. Researchers can specify M^3 of arbitrary complexity for novel task designs, as long as the model remains identified. The only requirement is that all activation formulas include the baseline activation b, which reflects random guessing if no other activations were included.

Tutorial 3 demonstrates how to define and fit a custom version, and Tutorial 4 shows how to validate custom specifications through parameter recovery simulations.

The bmm Workflow

Fitting M^3 with the *bmm* R package (Popov et al., 2025) follows a three-step workflow that is consistent for all measurement models implemented in the package:

1. **Define the model**: The package includes a constructor function *m3()* that generates a model object with all relevant information for model specification. This entails the variable names of response categories in the data, the number of response options per category, the model version, and the choice rule.
2. **Specify predictor formulas**: The *bmm* package provides a *bmmformula()* or shorthand *bmf()* function to specify the linear prediction formulas that define how experimental conditions and individual differences predict each activation parameter. The *bmmformula* syntax uses the same formula syntax as *brms* (Bürkner, 2017) and also includes random effects and non-linear formulas.[3]
3. **Fit the model**: The model object and prediction formula are passed to *bmm()* together with the data. This function translates the specification into a *brms* model and passes it to Stan (Carpenter et al., 2017) for Bayesian estimation.

Because *bmm* builds on *brms*, fitted models inherit most of the *brms* post-processing ecosystem, including posterior predictive checks, hypothesis tests, and model comparison.[4]

---

[3] For details on the *bmmformula* syntax, see https://venpopov.com/bmm/articles/bmm_bmmformula.html.

[4] We are continuously working on extending the post-processing functions to accommodate the specific requirements of the measurement models implemented in the *bmm* R package.

Tutorial 1: Simple Span M^3

This tutorial walks through the complete M^3 workflow for the *simple span* (ss) version using data from Oberauer (2019), who measured cued recall with words presented in boxes distributed horizontally on the screen. Two experiments (20 participants each) differed in stimulus set type: Experiment 1 used new words on each trial (open set), while Experiment 2 reused a small set of words across trials (closed set). Set size (2, 4, 6, 8 pairs) varied within participants. For more details on the experimental procedure please refer to the original publication. The full companion script available in the online supplement (`scripts/tutorial1_simple_span.R`) contains all code; we reproduce the key steps here.

Data

Each recognition trial presents a position probe ("?") in one of the boxes together with a set of response options. This set contained the correct target, other studied items from the same list, and not-presented lures. Participants selected one item, producing a categorical response: *correct*, *other* (intrusion from a different list position), or *NPL* (not-presented lure). The number of response options per category varied across 24 recognition conditions (combinations of set size, list items in the recognition set, and number of lures) that we will not model in detail here.

An M^3 requires aggregated data with one row per participant and condition. Each row contains (a) response frequency counts per category - how often the participant chose each option - and (b) the number of response options available per category. For the simple span version, this means three frequency columns (`corr`, `other`, `npl`) and three option-count columns (`n_corr`, `n_other`, `n_npl`), plus any predictors of interest (here, mean-centered set size `ss_lin` and experiment `exp`). The companion script (`scripts/tutorial1_simple_span.R`) shows how

to aggregate trial-level data from Oberauer (2019) into this format; the resulting dataset has 960 rows (40 participants × 24 recognition conditions). Below are the first rows of the data set:

```
# A tibble: 6 × 12
  id     exp   setsize rsizeList rsizeNPL  corr other   npl n_corr n_other n_npl
  <chr>  <fct>   <int>     <int>    <int> <int> <int> <int>  <dbl>   <dbl> <int>
1 10_cl… clos…       2         1        1    23     0     0      1       0     1
2 10_cl… clos…       2         2        0    23     0     0      1       1     0
3 10_cl… clos…       2         2        2    23     0     0      1       1     2
4 10_cl… clos…       4         1        1    23     0     0      1       0     1
5 10_cl… clos…       4         2        0    23     0     0      1       1     0
6 10_cl… clos…       4         2        2    22     0     1      1       1     2
# 1 more variable: ss_lin <dbl>
```

Model Specification

Specifying an M^3 requires two decisions: (a) which response categories and option counts define the task structure, and (b) how experimental conditions predict the activation parameters. In the *bmm* package, these are encoded via two objects - *m3()* for the model structure and *bmf()* for the predictor formulas. The simple span model for the data from Oberauer (2019) is specified as follows:

```
m3_model_ss <- m3(
  resp_cats   = c("corr", "other", "npl"),
  num_options = c("n_corr", "n_other", "n_npl"),
  version     = "ss",
  choice_rule = "softmax"
)
```

The simple span version estimates two parameters: a (item activation) and c (context binding). The baseline b is fixed for model identification (see Oberauer & Lewandowsky, 2019 for details). Note that pre-built versions assign activation formulas to categories by position, so `resp_cats` must follow the canonical order (for `ss`: `corr, other, npl`).[5]

[5] Misordering categories silently assigns the wrong activation formulas to the wrong categories, producing incorrect estimates without any warning.

With this information, we know that we can specify predictor formulas for the two activation parameters a and c. Because the two experiments in Oberauer (2019) are between-subjects, we use cell means coding (`0 + exp`) so that each experiment receives its own intercept rather than a difference from a reference level or grand mean. Similar to the original study by Oberauer (2019), we entered set size as a linear predictor (`exp:ss_lin`), and random effects allowed both intercepts and slopes to vary across participants, with separate variance components per experiment (`gr(id, by = exp)`)[6].

```
m3_formula <- bmf(
  c ~ 0 + exp + exp:ss_lin + (1 + ss_lin | gr(id, by = exp)),
  a ~ 0 + exp + exp:ss_lin + (1 + ss_lin | gr(id, by = exp))
)
```

Under the softmax rule, a and c are estimated on the identity link, so posterior draws are directly interpretable as activation values. The `bmm` package sets weakly informative default priors based on known parameter ranges; we use these defaults throughout Tutorial 1. Tutorial 2 shows how to set custom priors when needed.[7]

Model Fitting

The `bmm()` function translates the model specification to Stan code and estimates the posterior via MCMC (default: 4 chains, 2,000 iterations each, 1,000 warmup). We saved prior samples for later hypothesis tests and the full posterior for model comparisons via Bayes factors.[8]

---

[6] For details on the specification of the `bmmformula`, see the `bmm` package documentation and the documentation of the `brmsformula` function on which it builds.

[7] Default priors can be inspected with `default_prior(m3_formula, m3_model_ss, data = data_agg)` and overridden as in any `brms` model.

[8] Key arguments that specify this behavior are: `sample_prior = "yes"` stores prior samples for Savage-Dickey density ratios; `save_pars = save_pars(all = TRUE)` retains likelihood values

```r
fit_m3_ss_softmax <- bmm(
  formula      = m3_formula,
  model        = m3_model_ss,
  data         = data_agg,
  cores        = 4,
  backend      = "cmdstanr",
  sample_prior = "yes",
  save_pars    = save_pars(all = TRUE),
  file         = here("output", "fit_m3_ss_softmax")
)
```

After model fitting, convergence should be verified via $\hat{R}$ (Gelman & Rubin, 1992) and effective sample size. For all models in this tutorial, convergence criteria were met ($\hat{R} \leq 1.006$, bulk ESS > 514). If convergence issues arise (divergent transitions, low ESS), increasing iterations, adjusting `adapt_delta`, or simplifying the model are standard remedies.[9]

Model Evaluation

Before interpreting parameter estimates, we verify that the fitted model captures the observed data. A posterior predictive check compares observed response proportions to model-predicted proportions; systematic discrepancies would indicate that the activation formulas or choice rule fail to capture the data-generating process. We computed expected counts per category from the posterior predictive distribution, averaged across draws, and aggregated to participant-level proportions by set size and experiment (see the companion script `scripts/tutorial1_simple_span.R` for the full code). Figure 3 shows the result. Across all conditions and response categories, predicted proportions fall within one standard error of the

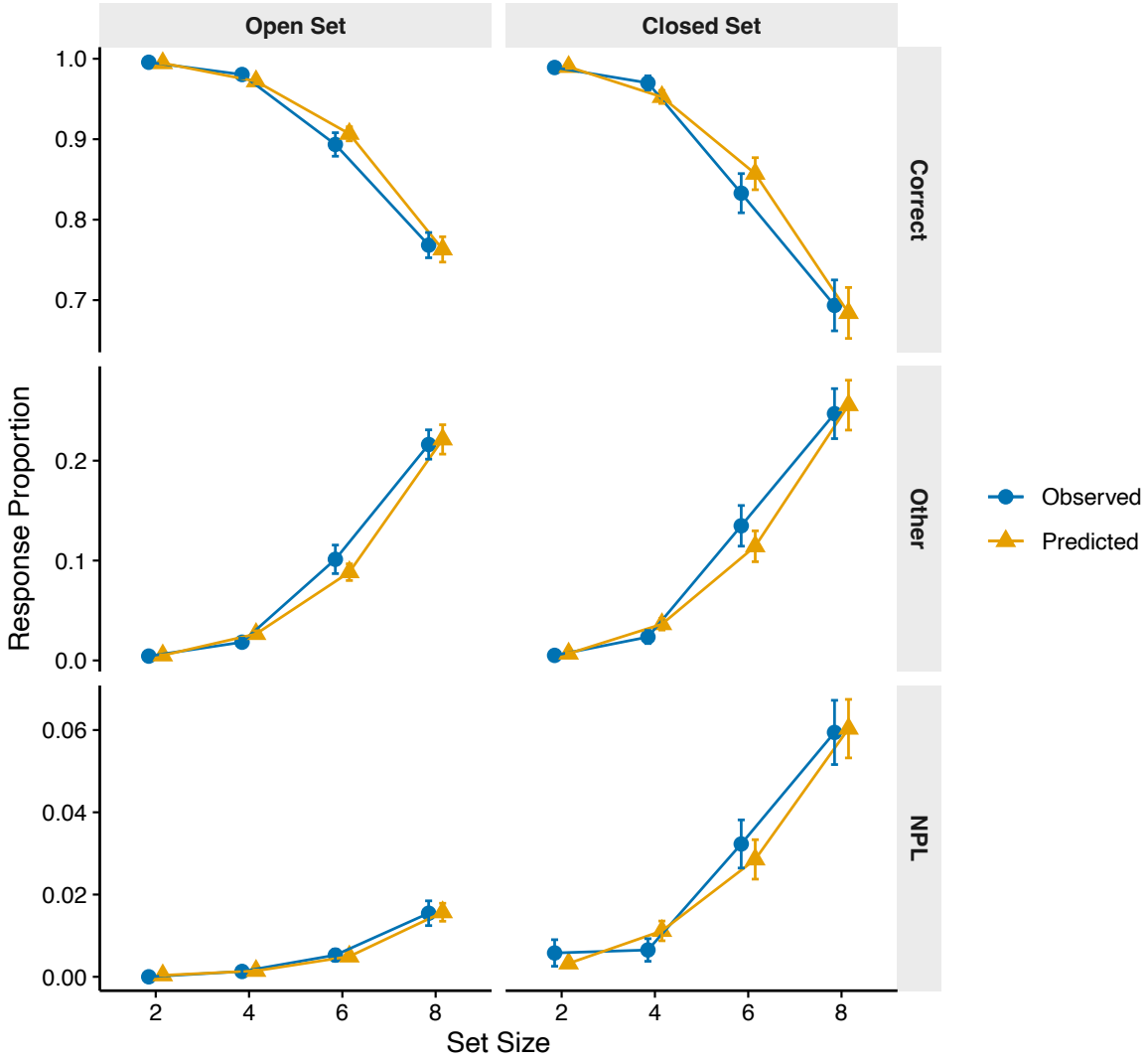
---

for bridge sampling; `file` caches the fitted model to disk. All arguments are passed to `brms::brm()` and can be modified as needed.

[9] M^3 uses the same Bayesian estimation machinery as any `brms` model, so standard convergence diagnostics apply and are provided via the `summary()` method. For guidance, see Gabry et al. (2019).

Figure 3

Posterior predictive check for the simple span M^3 (softmax choice rule). Points and lines show mean response proportions by set size and experiment; error bars show ± 1 SE across participants. Color distinguishes observed from predicted values.



observed means, indicating that the simple span M^3 with softmax choice rule adequately captures the response distributions.

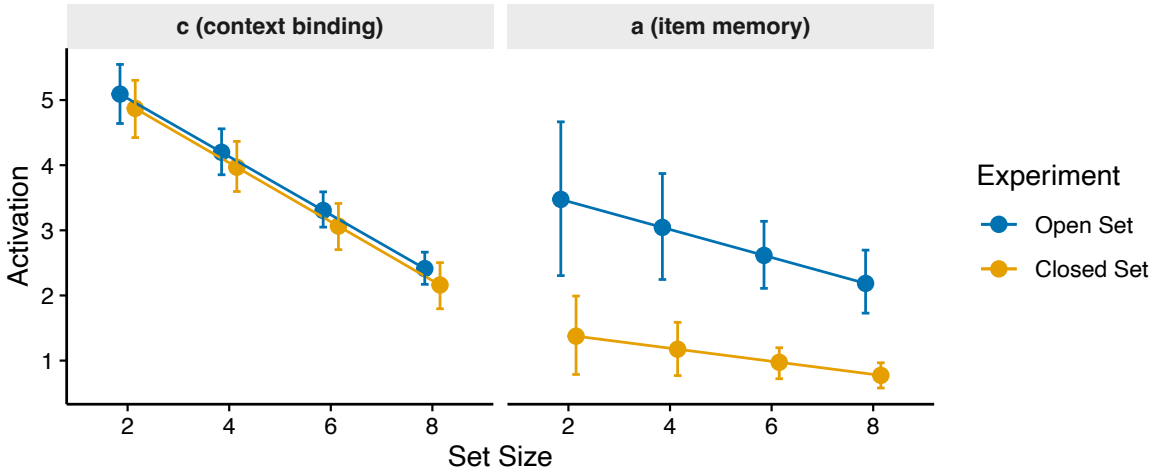
Parameter Estimates

With the model validated, we can examine how item memory (a) and context binding (c) varied across set sizes and experiments. Under the softmax choice rule, both parameters are on the identity link, so the posterior draws are directly interpretable as activation values. To compute condition-specific estimates, we reconstructed the linear predictor from the fixed effects: for each set size s, $\hat{a}_{\text{exp}} = \beta_{a,\text{exp}} + \beta_{a,\text{exp:ss\_lin}} \cdot (s - \bar{s})$, and analogously for c. The companion script extracts draws using `tidybayes::as_draws_df()` and computes 95% highest-density intervals [HDIs; McElreath (2020)] via `tidybayes::mean_hdci()`.

Figure 4 shows the resulting estimates. Context binding (c) is descriptively higher than item memory (a) across all set sizes in both experiments, with the gap being especially pronounced in the closed-set experiment. Both parameters decrease with set size, but the decline is substantially steeper for c than for a, consistent with binding being more sensitive to memory load. The two experiments differ markedly in item activation: a is notably higher in the open-set

Figure 4

Posterior estimates of item memory (a) and context binding (c) across set sizes for the open-set and closed-set experiments (softmax choice rule). Points show posterior means; error bars show 95% HDIs.



experiment than in the closed-set experiment. These results are consistent with the original findings by Oberauer (2019, see their discussion for theoretical interpretation of the experiment differences).

Choice Rule Comparison

As introduced in the M^3 framework section, the choice rule determines how activations are converted to response probabilities. Because the choice rule affects both the parameter scale and the model's fit to data, selecting the appropriate rule is an important modeling decision. To compare the two options, we fit a second model with `choice_rule = "simple"` using the same formula and data. Under the simple rule, a and c are estimated on the log link, so the raw coefficients must be exponentiated to obtain activation values.

The posterior predictive checks for both models are qualitatively similar, but the softmax rule tends to capture the tails of the response distribution more accurately, particularly for the NPL category at larger set sizes (see the posterior predictive plot in the online supplement: `figures/tutorial1_pp_check_simple.pdf`). The simple rule slightly overpredicts other-list intrusions in some conditions.[10]

For a formal comparison, we computed the marginal likelihood of each model via bridge sampling (Gronau et al., 2017) and derive a Bayes factor. This requires to set `save_pars(all = TRUE)` during fitting (set in the `bmm()` call above):

```
bridge_softmax <- bridge_sampler(fit_m3_ss_softmax, repetitions = 10)
bridge_simple  <- bridge_sampler(fit_m3_ss_simple,  repetitions = 10)
bf(bridge_softmax, bridge_simple)
```

[10] The companion script (`scripts/tutorial1_simple_span.R`) generates posterior predictive figures for both models; readers can inspect these in the online supplement to compare the fit of both choice rules across all conditions.

The Bayes factor strongly favored the softmax rule (median $BF_{10} = 4.8 \times 10^7$ across 10 bridge sampling repetitions). This is consistent with recent comparisons by Oberauer (2023) and Li et al. (2026), who found the softmax choice rule to outperform the simple rule across different working memory tasks. We use the softmax rule as the default in the remaining tutorials.

Hypothesis Testing

Because we fitted the model with `sample_prior = "yes"`, we can use `brms::hypothesis()` to test directed hypotheses and obtain evidence ratios (Savage-Dickey density ratios comparing posterior to prior density at the hypothesis boundary). For example, to test whether set size reduces context binding (averaged across experiments):

```
# Set size effect on context binding (averaged across experiments)
hypothesis(fit_m3_ss_softmax,
  "(c_expopenset:ss_lin + c_expclosedset:ss_lin) / 2 = 0")
```

This returns a Savage-Dickey density ratio: the ratio of prior to posterior density at the point null. Values substantially greater than 1 indicate that the data have shifted belief away from equality; values near 1 indicate inconclusive evidence. There is strong evidence for c decreasing with larger set size ($BF_{10} = \geq 10{,}000$). We can similarly test whether set size reduces item memory and whether the set size effects differ between experiments:

```
# Set size effect on item memory (averaged across experiments)
hypothesis(fit_m3_ss_softmax,
  "(a_expopenset:ss_lin + a_expclosedset:ss_lin) / 2 = 0")

# Experiment difference in set size effect on c
hypothesis(fit_m3_ss_softmax,
  "c_expopenset:ss_lin = c_expclosedset:ss_lin")
```

There is also moderate evidence that set size reduces a (BF_{10} = 5.61). The set size effect on c is comparable across experiments (BF_{01} = 62.8). The companion script contains the full set of hypothesis tests.

Tutorial 2: Complex Span M^3

This tutorial extends the workflow to tasks where distractors appear alongside memoranda. We use data from Li et al. (2026), who developed a complex-span filtering task to test whether processed information can be kept out of working memory. The companion script is `scripts/tutorial2_complex_span.R`; data preparation is in `scripts/prepare_Li2026_data.R`.

What makes complex span different

Complex span tasks impose higher cognitive load than simple span tasks because they require participants to perform a processing task and a memory task simultaneously (for a detailed description and task illustration, see Li et al., 2026, Figure 1). In these tasks, stimuli that only serve the processing task but not the memory task may be encoded into working memory and interfere with the maintenance of other items. These stimuli are referred to as distractors and may also be recalled at retrieval. Therefore, in addition to the three response categories used in simple span tasks, complex span tasks include two additional categories to account for distractor responses: *paired distractor*, which refers to the distractor associated with the context of the tested target, and *other distractor*, which refers to distractors from different contexts. In the complex span (`cs`) model, a filtering process (represented by parameter f) determines the extent to which a distractor is activated relative to a memorandum. In the model for complex span tasks

proposed by Oberauer and Lewandowsky (2019) it appears as a multiplier on item memory and context binding for distractor categories:

$$
\begin{aligned}
A_{\text{correct}} &= b + a + c \\
A_{\text{close dist.}} &= b + f \cdot a + f \cdot c \\
A_{\text{other}} &= b + a \\
A_{\text{other dist.}} &= b + f \cdot a \\
A_{\text{NPL}} &= b
\end{aligned}
$$

The parameter f ranges from 0 to 1 (estimated on the logit scale). At the extremes, $f = 0$ means distractors receive only baseline activation, as if they were never encountered (perfect filtering); $f = 1$ means distractors are activated identically to memoranda, as if filtering failed entirely.

Data

Li et al. (2026) presented participants with image pairs and manipulated whether targets and distractors were identified using an arrow cue pointing to one of the items before processing (pre-cue), after processing (retro-cue), or whether no distractors were present (control). We use Experiment 1 (45 participants after exclusions, within-subjects) for this tutorial. The data are pre-aggregated into response frequencies across the five categories:

```
# A tibble: 6 × 12
  participant condition  corr other distc disto   npl n_corr n_other n_distc
        <dbl> <fct>     <dbl> <dbl> <dbl> <dbl> <dbl>  <dbl>   <dbl>   <dbl>
1           1 control      41    18     0     0     1      1       2       0
2           1 pre          31    17     4     6     1      1       2       1
3           1 retro        34    15     5     4     1      1       2       1
4           2 control      52     7     0     0     1      1       2       0
5           2 pre          33    14     6     7     0      1       2       1
6           2 retro        44     9     4     3     0      1       2       1
# 2 more variables: n_disto <dbl>, n_npl <dbl>
```

Model specification

The model specification follows the same pattern as Tutorial 1, now with five response categories in their canonical order (`corr, distc, other, disto, npl`):

```
m3_model_cs <- m3(
  resp_cats   = c("corr", "distc", "other", "disto", "npl"),
  num_options = c("n_corr", "n_distc", "n_other", "n_disto", "n_npl"),
  version     = "cs",
  choice_rule = "softmax"
)
```

The `cs` version estimates three parameters: a (item activation), c (context binding), and f (filtering). The predictor formula specifies condition effects on all three with cell means coding and uncorrelated random effects:

```
m3_formula_cs <- bmf(
  c ~ 0 + condition + (0 + condition || participant),
  a ~ 0 + condition + (0 + condition || participant),
  f ~ 0 + condition + (0 + condition || participant)
)
```

In the control condition, no distractors are presented, so f does not influence the likelihood and is not identified. We fix f in the control condition at a constant value to prevent the sampler from wandering through the prior:

```
priors_cs <- c(
  prior(constant(0), nlpar = "f", coef = "conditioncontrol"),
  prior(constant(0), class = "sd", nlpar = "f",
        coef = "conditioncontrol", group = "participant")
)
```

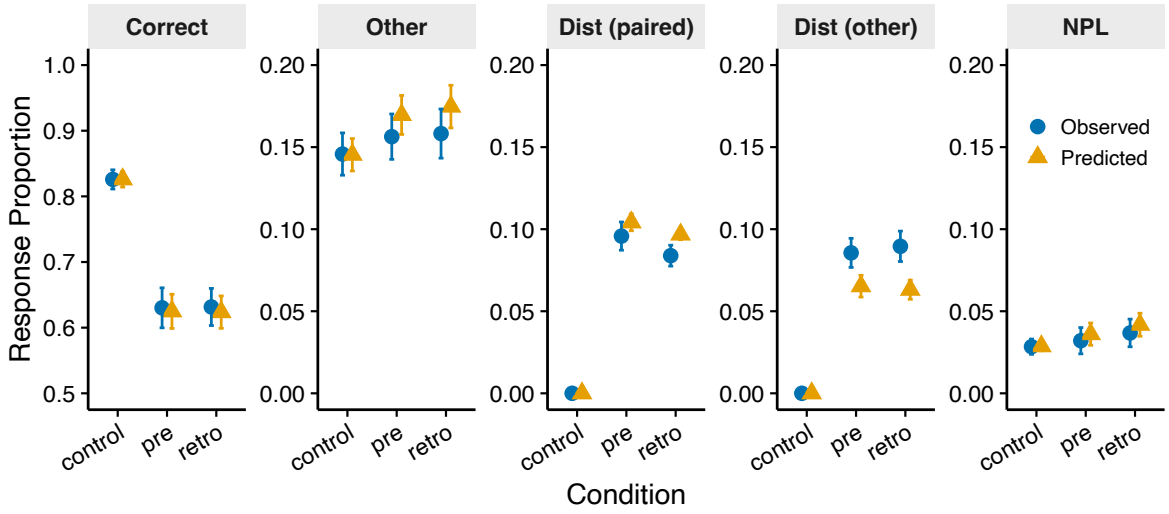
Because f does not affect the likelihood in the control condition, the specific fixed value is not critical. Setting the fixed effect to 0 on the logit scale and the random effect SD to 0 pins control-condition filtering at a constant. The remaining conditions estimate f freely. The companion script contains the full prior specification.

Model fitting and evaluation

Fitting follows the same `bmm()` call as in Tutorial 1, now additionally passing the custom priors. The model converged without issues ($\hat{R} \leq 1.005$, bulk ESS $> 1{,}027$, no divergent

Figure 5

Posterior predictive check for the complex span M^3. Points show mean response proportions per category and condition; error bars show ± 1 SE across participants.



transitions). Figure 5 shows the posterior predictive check. The model captured the observed response proportions well across all three conditions and five categories.

Parameter estimates and interpreting f

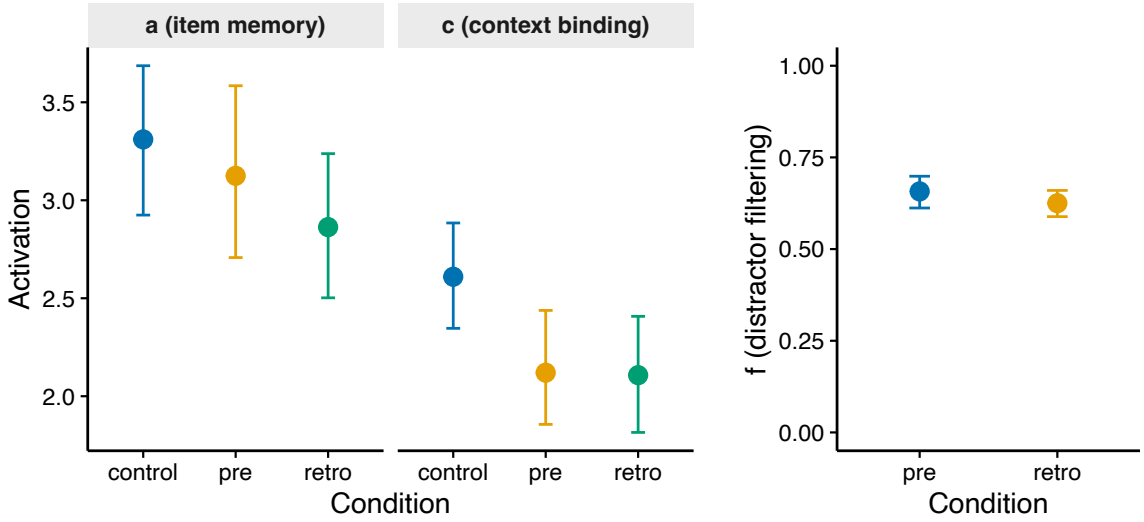
We now examine the posterior estimates for all three parameters. Item memory (a) and context binding (c) are on the identity link and can be read directly from the posterior draws. The filtering parameter f, however, is estimated on the logit scale. To interpret it as a proportional reduction between 0 and 1, we apply the inverse logit transformation (`plogis()`):

```
draws <- as_draws_df(fit_m3_cs) %>% as_tibble()
f_pre   <- plogis(draws$b_f_conditionpre)
f_retro <- plogis(draws$b_f_conditionretro)
```

On the probability scale, f is around 0.65 in both conditions (pre-cue: M = 0.66, 95% HDI [0.61, 0.70]; retro-cue: M = 0.63, 95% HDI [0.59, 0.66]), indicating some filtering (see Figure 6). This matches the conclusion of Li et al. (2026): once distractors are processed, they

Figure 6

Posterior parameter estimates for the complex span M^3. Left panels show item memory (a) and context binding (c) across conditions. Right panel shows distractor filtering (f) for pre-cue and retro-cue conditions, transformed to the probability scale. The control condition is omitted from the f panel because it was fixed (no distractors present).



are encoded into working memory regardless of whether participants knew in advance which items were distractors.

Hypothesis tests

Again, we can test condition differences for the different parameters formally using *brms::hypothesis()*:

```
# Does f differ between pre-cue and retro-cue?
hypothesis(fit_m3_cs, "f_conditionpre - f_conditionretro = 0")

# Do a and c differ between control and distractor conditions?
hypothesis(fit_m3_cs,
  "a_conditioncontrol - (a_conditionpre + a_conditionretro) / 2 = 0")
hypothesis(fit_m3_cs,
  "c_conditioncontrol - (c_conditionpre + c_conditionretro) / 2 = 0")
```

The evidence ratio for the f comparison indicated that filtering is equivalent between cue conditions ($BF_{01} = 10.2$). There is inconclusive evidence for item memory a comparing the distractor conditions to the control ($BF_{10} = 0.45$); however, context activation c is lower in the distractor conditions ($BF_{10} = 7.73$).

Tutorial 3: Custom M^3

The previous tutorials used the `ss` and `cs` specifications proposed by Oberauer and Lewandowsky (2019), which come with fixed activation formulas. However, these pre-built versions impose constraints that may not match every research question. For example, the `cs` version assumes symmetric distractor attenuation: both item memory and context binding are scaled by the single parameter f. This rules out the possibility that item and context information transfer to distractors at different rates.

The original analysis by Li et al. (2026) tested this distinction using a custom specification with two separate ratio parameters, r_a and r_c, and found that item memory transfers almost fully to distractors while context binding is more attenuated. When a research question requires this kind of flexibility - or when the task design does not map onto the simple or complex span versions - the M^3 framework supports fully custom specifications with user-defined response categories and activation formulas. This tutorial demonstrates how to define, fit, and compare such custom specifications using the same data as Tutorial 2. The companion script is `scripts/tutorial3_custom_filtering.R`.

From single f to separate ratio parameters

Replacing f with two ratio parameters - r_a for item memory and r_c for context binding - produces the following activation formulas:

$$\begin{aligned} A_{\text{correct}} &= b + a + c \\ A_{\text{close dist.}} &= b + r_a \cdot a + r_c \cdot c \\ A_{\text{other}} &= b + a \\ A_{\text{other dist.}} &= b + r_a \cdot a \\ A_{\text{NPL}} &= b \end{aligned}$$

When $r_a = r_c$, this reduces to the standard `cs` version with $f = r_a = r_c$. The custom version relaxes this constraint and lets the data decide.

Defining a custom M^3

Defining a custom M^3 requires specifying the response categories, the number of options per category, and — importantly — the link functions for each parameter to ensure they are on the appropriate scale. Because r_a and r_c are proportions bounded between 0 and 1, we use the logit link; a and c are unbounded activations and use the identity link:

```
m3_model_custom <- m3(
  resp_cats   = c("corr", "distc", "other", "disto", "npl"),
  num_options = c("n_corr", "n_distc", "n_other", "n_disto", "n_npl"),
  version     = "custom",
  choice_rule = "softmax",
  links = list(
    a  = "identity", c  = "identity",
    ra = "logit",    rc = "logit"
  )
)
```

For custom versions, `bmf()` specifies both the activation formulas (how parameters combine into category activations) and the regression formulas (how experimental conditions

predict each parameter). The activation formulas translate the equations above into R formula syntax:

```
m3_formula_custom <- bmf(
  corr  ~ a + c + b,
  distc ~ ra * a + rc * c + b,
  other ~ a + b,
  disto ~ ra * a + b,
  npl   ~ b,
  a  ~ 0 + condition + (0 + condition || participant),
  c  ~ 0 + condition + (0 + condition || participant),
  ra ~ 0 + condition + (0 + condition || participant),
  rc ~ 0 + condition + (0 + condition || participant)
)
```

As in Tutorial 2, r_a and r_c are not identified in the control condition (no distractors present), so we fix them with constant priors on both the fixed effect and the random effect SD:

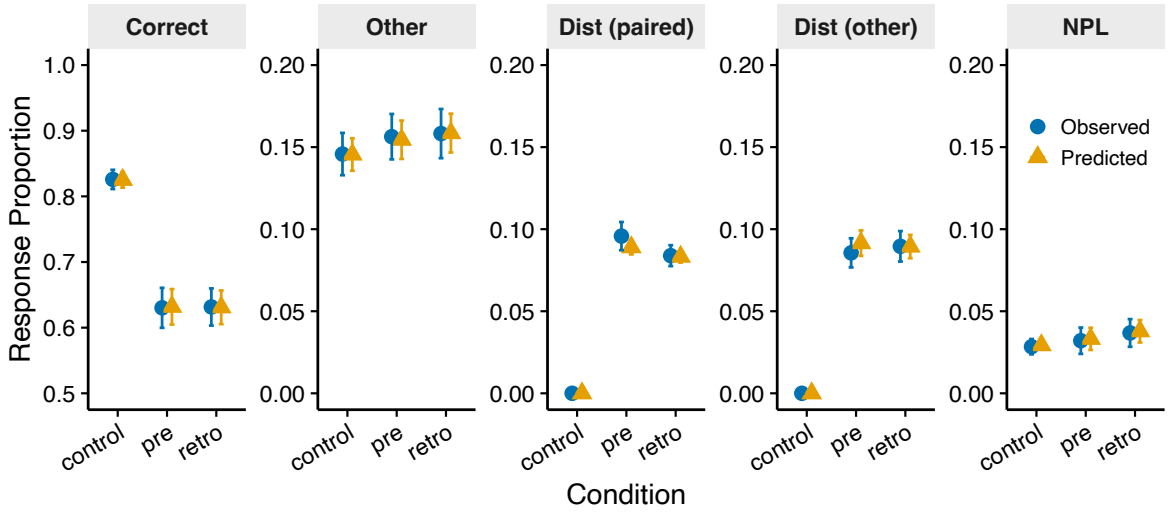
```
priors_custom <- c(
  prior(constant(-10), nlpar = "ra", coef = "conditioncontrol"),
  prior(constant(0),   class = "sd", nlpar = "ra",
        coef = "conditioncontrol", group = "participant"),
  prior(constant(-10), nlpar = "rc", coef = "conditioncontrol"),
  prior(constant(0),   class = "sd", nlpar = "rc",
        coef = "conditioncontrol", group = "participant")
)
```

Model fitting and evaluation

Fitting follows the same `bmm()` call as in the previous tutorials. The model converged without issues ($\hat{R} \leq 1.007$, bulk ESS > 794, no divergent transitions). Figure 7 shows the posterior predictive check. Predicted proportions closely match the observed data across all conditions and categories. Compared to the `cs` fit in Tutorial 2, the custom version captures the error categories (distractor and other intrusions) more precisely, reflecting the additional flexibility of separate ratio parameters for item memory and context binding.

Figure 7

Posterior predictive check for the custom filtering M^3. Points show mean response proportions per category and condition; error bars show ± 1 SE across participants.



Model comparison

To evaluate whether separating r_a and r_c improves on the standard `cs` version, we compared the two using bridge sampling (Gronau et al., 2017):

```
bridge_cs     <- bridge_sampler(fit_m3_cs, repetitions = 10)
bridge_custom <- bridge_sampler(fit_m3_custom, repetitions = 10)
bf(bridge_custom, bridge_cs)
```

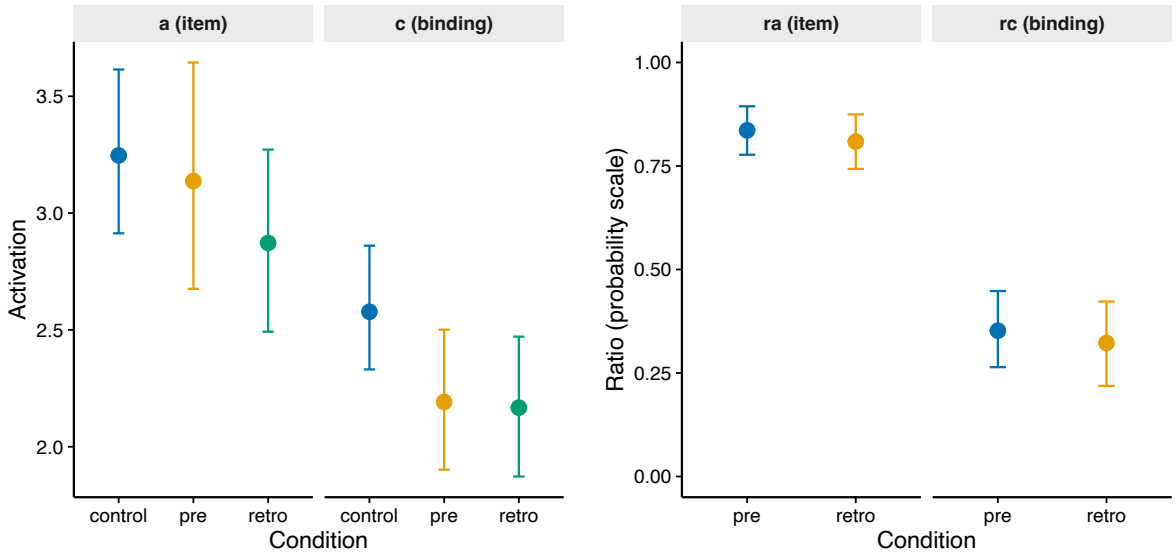
The Bayes factor strongly favored the custom version (median $\text{BF}_{10} = 6.4 \times 10^{17}$ across 10 bridge sampling repetitions), indicating that the data warrant separate ratio parameters rather than a single f.

Parameter estimates

Figure 8 shows the posterior estimates. Item memory (a) and context binding (c) are similar to the `cs` estimates from Tutorial 2. The ratio parameters reveal an asymmetry: on the probability scale, r_a is high in both conditions (pre-cue: $M = 0.84$, 95% HDI [0.78, 0.89]; retro-

Figure 8

Posterior parameter estimates for the custom filtering M^3. Left panels show item memory (a) and context binding (c) across conditions. Right panels show the ratio parameters r_a (item) and r_c (binding) for pre-cue and retro-cue conditions, transformed to the probability scale.



cue: M = 0.81, 95% HDI [0.74, 0.87]), meaning item memory transfers almost fully to distractors. r_c is substantially lower (pre-cue: M = 0.35, 95% HDI [0.26, 0.45]; retro-cue: M = 0.32, 95% HDI [0.22, 0.42]), meaning context binding is more attenuated for distractors. This pattern matches the published findings of Li et al. (2026).

Hypothesis tests

As in Tutorial 2, we can test parameter differences formally. Within both the pre-cue (BF_{10} = ≥ 10,000) and retro-cue condition (BF_{10} = ≥ 10,000), there is strong evidence that $r_a > r_c$. Across conditions, r_a does not differ between pre-cue and retro-cue (BF_{01} = 6.32), and the same holds for r_c (BF_{01} = 7.63). Item memory transfers to distractors regardless of cue timing, while context binding is selectively attenuated, consistent with the view that filtering operates on binding rather than item activation.

Tutorial 4: Parameter Recovery

The M^3 framework allows researchers to develop custom versions tailored to their research questions. However, before applying a custom M^3 to empirical data, it is worth checking whether the planned design provides enough information to recover the parameters of interest. If recovery fails when the generating values are known and the model is correctly specified, the problem lies with the design or the specification, not the data. This tutorial demonstrates the recovery workflow using the custom filtering version from Tutorial 3. The companion script is `scripts/tutorial4_parameter_recovery.R`.

When to assess parameter recovery

Parameter recovery is not mandatory for every analysis, but it is informative as part of a principled Bayesian workflow (Schad et al., 2021; Wilson & Collins, 2019) — particularly when using a custom specification for the first time, when the design has unusual trial counts, or when individual-level estimates will be used for correlations. Beyond validating a given design, recovery simulations can also inform experimental planning: by systematically varying the number of participants or trials, researchers can determine the sample size and trial count needed to achieve acceptable recovery for their parameters of interest. Even a single recovery run is practically feasible (one to two hours of computation) and reveals how well the design can distinguish the parameters of interest (for a systematic treatment, see Göttmann et al., 2025).

Simulation workflow

We simulate data from the custom filtering version (Tutorial 3) with known parameter values and then fit the same specification to the simulated data. For each of the four parameters

Table 1

Generating parameter values for the recovery simulation. All values are on the link scale.

Parameter	Link	Mean (link)	SD (link)	Native range (±2 SD)
a	identity	1.5	0.50	0.50 – 2.50
c	identity	2.0	0.50	1.00 – 3.00
r_a	logit	0.85	0.40	0.51 – 0.84
r_c	logit	0.25	0.40	0.37 – 0.74

(a, c, r_a, r_c), we specify group-level means and between-person standard deviations on the link scale. Table 1 lists the generating values, chosen to approximate the empirical estimates from Li et al. (2026).

Individual parameters are drawn from $\mathcal{N}(\mu_{\text{link}}, \sigma_{\text{link}})$ independently for each person and parameter. We simulated $N = 50$ participants with 60 trials (i.e., 60 response observations) per participant. The *bmm* package provides *rm3()* as the data-generating counterpart to the fitting workflow:

```
demo_data <- rm3(
  n = 1, size = 60,
  pars = c(a = 1.5, c = 2.0, ra = 0.70, rc = 0.56),
  m3_model = m3_model_custom,
  act_funs = m3_act_funs
)
```

This returns a one-row data frame with response counts across the five categories. To simulate a full dataset, we apply *rm3()* over all 50 participants, producing 50 rows. The companion script contains the complete generation and fitting code.

Recovery evaluation

Group-level recovery

Figure 9 shows the group-level results. All 4 of 4 population means fell within their 95% credible intervals, and 4 of 4 between-person SDs were recovered accurately. Bias was negligible for all parameters. The companion script extracts these values from `fixef()` and `brms::VarCorr()`.

Individual-level recovery

Figure 10 shows scatter plots of true versus recovered individual parameters ($N = 50$, 60 retrievals per participant). Context binding (c) was recovered well ($r = 0.92$), and item memory

Figure 9

Group-level parameter recovery. Points show recovered posterior medians; error bars show 95% credible intervals. The dashed line indicates perfect recovery.

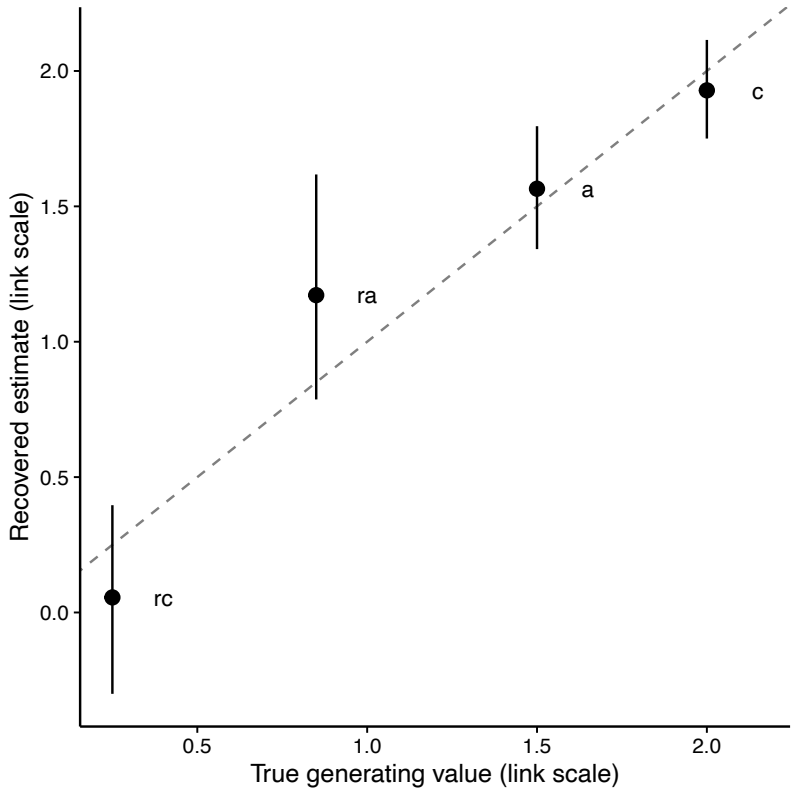
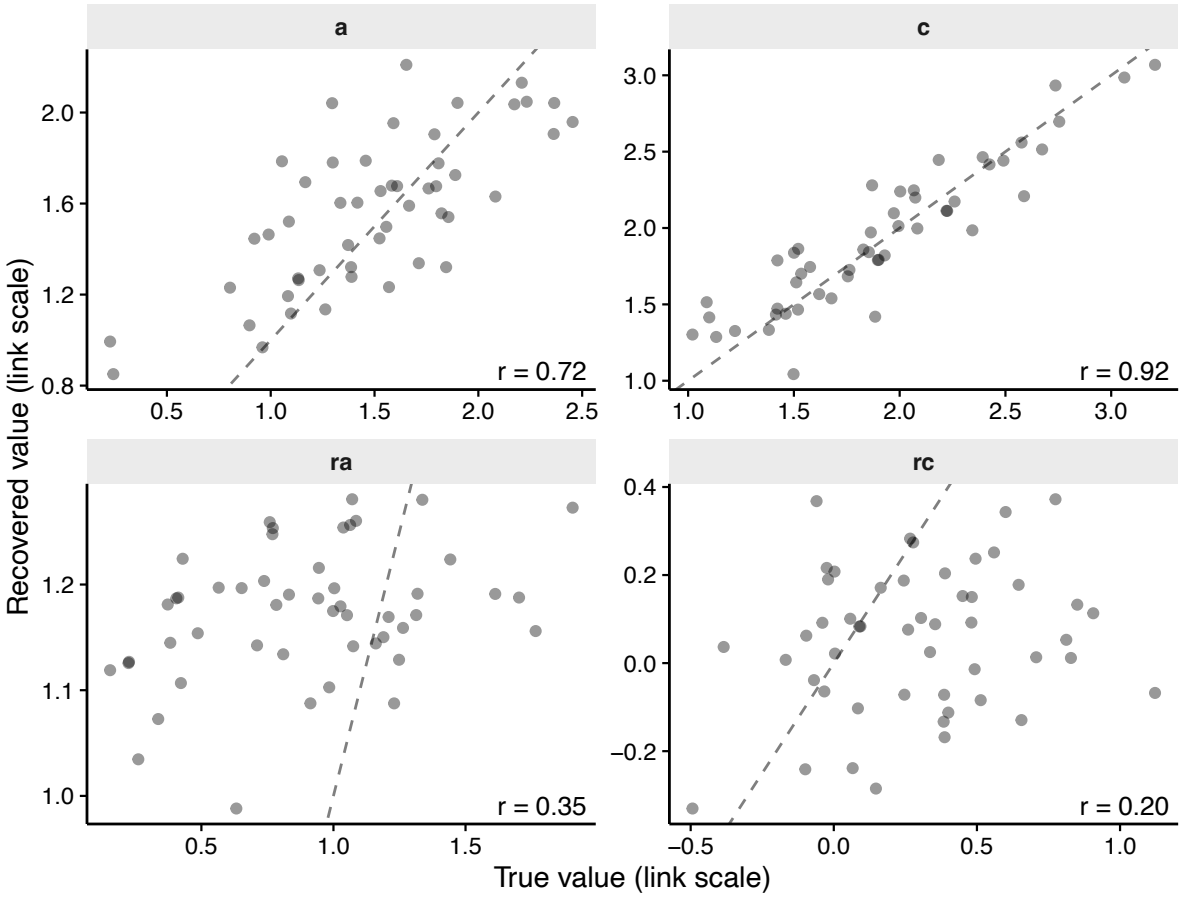


Figure 10

Individual-level parameter recovery ($N = 50$, 60 retrievals per participant). Each point is one participant; the dashed line indicates perfect recovery. Pearson correlations are shown per parameter.



(a) showed adequate recovery (r = 0.72). The ratio parameters were harder to recover individually (r_a: r = 0.35; r_c: r = 0.20). This is expected: ratio parameters appear only in distractor categories, which have lower base rates and therefore carry less information per trial.

Practical guidance

As a rule of thumb, if individual-level correlations fall below .7 for a parameter, the design likely needs more trials or a larger sample before that parameter can support individual-

differences analyses. The present simulation uses a single replication; a full simulation study should vary sample size and trial count systematically (for a systematic approach, see Göttmann et al., 2025; for implementation tools, see Chalmers & Adkins, 2020; for a complementary calibration-based approach, see Talts et al., 2018).

Going further: more complex custom versions

The filtering example in Tutorial 3 modifies only the activation formulas of the distractor categories, keeping the same five-category structure as the `cs` version. Custom specifications can go much further: entirely new response categories, nonlinear activation formulas with design variables, and additional parameters with different link functions. Two considerations are important when designing custom specifications. First, the activation formulas should be theoretically motivated - each parameter should correspond to a meaningful cognitive process, and the way parameters combine into category activations should reflect plausible assumptions about the task. Second, all parameters must be identifiable given the data structure. Identification requires that the number of response categories and experimental conditions jointly provide enough constraints to distinguish each parameter. When parameters are not identified, the posterior will reflect the prior rather than the data, and parameter recovery will fail.

The Appendix presents a more complex example, a memory updating task from Oberauer and Lewandowsky (2019) with five estimated parameters and experimentally manipulated timing variables, to illustrate how design variables can provide the additional constraints needed for more complex specifications. The accompanying scripts (`scripts/tutorial5_parameter_recovery_updating_simple.R` and

scripts/tutorial5_parameter_recovery_updating.R) provide the complete code for this advanced example.

Summary and Additional Resources

This tutorial covered four progressively complex applications of the M^3 framework using the *bmm* R package. The simple span example (Tutorial 1) introduced the full workflow — specification, fitting, evaluation, and hypothesis testing — and compared the two choice rules. The complex span example (Tutorial 2) added distractor categories, the filtering parameter f, and the handling of non-identified parameters. The custom specification example (Tutorial 3) showed how to relax assumptions of a pre-built version by defining separate ratio parameters for item memory and context binding, and compared the custom version against the standard `cs` version. The parameter recovery tutorial (Tutorial 4) demonstrated how to validate a custom specification through simulation before applying it to empirical data.

A few practical recommendations. If the task produces three response categories (correct, other-list intrusion, NPL), the simple span version (`ss`) is the natural starting point. Tasks with distractors that are the same material type as memoranda can fit the complex span version (`cs`). For other designs, the custom version gives full control over activation formulas, parameters, and link functions. In all cases, the softmax choice rule is the recommended default based on current evidence. When defining a custom specification, running a parameter recovery simulation before data collection is good practice: it reveals whether the planned design can distinguish the parameters of interest and whether the specification is identified.

We hope that the implementation of the M^3 framework in the *bmm* package and the accompanying tutorials will make this modeling approach accessible to a wide range of

researchers. As noted before, M^3 is not limited to memory tasks but can be applied to any decision paradigm yielding a multinomial response distribution (Busemeyer & Townsend, 1993; McFadden, 1974; Roe et al., 2001). We encourage users to explore the package documentation, experiment with different model specifications, and contribute to the ongoing development of this tool.

Additional resources

- M^3 framework paper: Oberauer and Lewandowsky (2019)
- Systematic parameter recovery for M^3: Göttmann et al. (2025)
- Companion *bmm* tutorial for continuous-response mixture models: Frischkorn and Popov (2025)
- *bmm* package documentation and M^3 vignette: https://venpopov.github.io/bmm/articles/bmm_m3.html
- OSF repository with data, code, and materials: https://osf.io/yb7wm/
- Full companion scripts for all four tutorials on GitHub

References

Batchelder, W. H., & Riefer, D. M. (1999). Theoretical and empirical review of multinomial process tree modeling. *Psychonomic Bulletin & Review*, *6*(1), 57–86. https://doi.org/10.3758/BF03210812

Bürkner, P.-C. (2017). brms: An R package for Bayesian multilevel models using Stan. *Journal of Statistical Software*, *80*(1), 1–28. https://doi.org/10.18637/jss.v080.i01

Busemeyer, J. R., & Townsend, J. T. (1993). Decision field theory: A dynamic-cognitive approach to decision making in an uncertain environment. *Psychological Review*, *100*(3), 432–459. https://doi.org/10.1037/0033-295X.100.3.432

Carpenter, B., Gelman, A., Hoffman, M. D., Lee, D., Goodrich, B., Betancourt, M., Brauer, M., Guo, J., Li, P., & Riddell, A. (2017). Stan: A probabilistic programming language. *Journal of Statistical Software*, *76*(1), 1–32. https://doi.org/10.18637/jss.v076.i01

Chalmers, R. P., & Adkins, M. C. (2020). Writing effective and reliable Monte Carlo simulations with the SimDesign package. *The Quantitative Methods for Psychology*, *16*(4), 248–280. https://doi.org/10.20982/tqmp.16.4.p248

Erdfelder, E., Auer, T.-S., Hilbig, B. E., Aßfalg, A., Moshagen, M., & Nadarevic, L. (2009). Multinomial processing tree models: A review of the literature. *Zeitschrift für Psychologie / Journal of Psychology*, *217*(3), 108–124. https://doi.org/10.1027/0044-3409.217.3.108

Frischkorn, G. T., & Popov, V. (2025). A tutorial for estimating Bayesian hierarchical mixture models for visual working memory tasks: Introducing the Bayesian Measurement Modeling (bmm) package for R. *Behavior Research Methods*, *57*(5), 144. https://doi.org/10.3758/s13428-025-02643-0

Gabry, J., Simpson, D., Vehtari, A., Betancourt, M., & Gelman, A. (2019). Visualization in Bayesian workflow. *Journal of the Royal Statistical Society: Series A (Statistics in Society)*, *182*(2), 389–402. https://doi.org/10.1111/rssa.12378

Gelman, A., & Rubin, D. B. (1992). Inference from iterative simulation using multiple sequences. *Statistical Science*, *7*(4), 457–472. https://doi.org/10.1214/ss/1177011136

Göttmann, J., Frischkorn, G. T., Oberauer, K., Schaefer, S. B., & Schubert, A.-L. (2025). *Modeling individual differences in working memory: Subject-level parameter recovery within the Memory Measurement Model framework (M3)*. https://doi.org/10.31234/osf.io/945d2

Gronau, Q. F., Sarafoglou, A., Matzke, D., Ly, A., Boehm, U., Marsman, M., Leslie, D. S., Forster, J. J., Wagenmakers, E.-J., & Steingroever, H. (2017). A tutorial on bridge sampling. *Journal of Mathematical Psychology*, *81*, 80–97. https://doi.org/10.1016/j.jmp.2017.09.005

Heck, D. W., Arnold, N. R., & Arnold, D. (2018). TreeBUGS: An R package for hierarchical multinomial-processing-tree modeling. *Behavior Research Methods*, *50*(1), 264–284. https://doi.org/10.3758/s13428-017-0869-7

Jacoby, L. L. (1991). A process dissociation framework: Separating automatic from intentional uses of memory. *Journal of Memory and Language*, *30*(5), 513–541. https://doi.org/10.1016/0749-596X(91)90025-F

Li, C., Frischkorn, G. T., Dames, H., & Oberauer, K. (2025). The benefit of removing information from working memory: Increasing available cognitive resources or reducing interference? *Cognition*, *260*. https://doi.org/10.1016/j.cognition.2025.106134

Li, C., Frischkorn, G. T., & Oberauer, K. (2025). Updating of information in working memory: Time course and consequences. *Cognitive Psychology*, *156*, 101702. https://doi.org/10.1016/j.cogpsych.2024.101702

Li, C., Frischkorn, G. T., & Oberauer, K. (2026). Can we process information without encoding it into working memory? *Journal of Experimental Psychology: Learning, Memory, and Cognition*. https://doi.org/10.1037/xlm0001585

Loaiza, V. M., & Souza, A. S. (2024). Active maintenance in working memory reinforces bindings for future retrieval from episodic long-term memory. *Memory & Cognition*, *52*(8), 1999–2021. https://doi.org/10.3758/s13421-024-01596-7

Luce, R. D. (1959). *Individual choice behavior: A theoretical analysis*. John Wiley & Sons.

McElreath, R. (2020). *Statistical rethinking: A Bayesian course with examples in R and Stan* (2nd ed.). Chapman; Hall/CRC. https://doi.org/10.1201/9780429029608

McFadden, D. (1974). Conditional logit analysis of qualitative choice behavior. In P. Zarembka (Ed.), *Frontiers in econometrics* (pp. 105–142). Academic Press.

Oberauer, K. (2019). Working memory capacity limits memory for bindings. *Journal of Cognition*, *2*(1), 40. https://doi.org/10.5334/joc.86

Oberauer, K. (2023). Measurement models for visual working memory—A factorial model comparison. *Psychological Review*, *130*(3), 841–852. https://doi.org/10.1037/rev0000328

Oberauer, K., & Lewandowsky, S. (2019). Simple measurement models for complex working-memory tasks. *Psychological Review*, *126*(6), 880–932. https://doi.org/10.1037/rev0000159

Oberauer, K., Lewandowsky, S., Awh, E., Brown, G. D. A., Conway, A., Cowan, N., Donkin, C., Farrell, S., Hitch, G. J., Hurlstone, M. J., Ma, W. J., Morey, C. C., Nee, D. E.,

Schweppe, J., Vergauwe, E., & Ward, G. (2018). Benchmarks for models of short-term and working memory. *Psychological Bulletin*, *144*(9), 885–958. https://doi.org/10.1037/bul0000153

Popov, V., Frischkorn, G. T., Li, C., & Bürkner, P.-C. (2025). *Bmm: Easy and accessible Bayesian measurement models using 'brms'*. https://doi.org/10.32614/CRAN.package.bmm

Roe, R. M., Busemeyer, J. R., & Townsend, J. T. (2001). Multialternative decision field theory: A dynamic connectionist model of decision making. *Psychological Review*, *108*(2), 370–392. https://doi.org/10.1037/0033-295X.108.2.370

Schad, D. J., Betancourt, M., & Vasishth, S. (2021). Toward a principled Bayesian workflow in cognitive science. *Psychological Methods*, *26*(1), 103–126. https://doi.org/10.1037/met0000275

Schubert, A.-L., Löffler, C., Sadus, K., Göttmann, J., Hein, J., Schröer, P., Teuber, A., & Hagemann, D. (2023). Working memory load affects intelligence test performance by reducing the strength of relational item bindings and impairing the filtering of irrelevant information. *Cognition*, *236*, 105438. https://doi.org/10.1016/j.cognition.2023.105438

Talts, S., Betancourt, M., Simpson, D., Vehtari, A., & Gelman, A. (2018). *Validating Bayesian inference algorithms with simulation-based calibration*. https://arxiv.org/abs/1804.06788

Thurstone, L. L. (1927). A law of comparative judgment. *Psychological Review*, *34*(4), 273–286. https://doi.org/10.1037/h0070288

Wilson, R. C., & Collins, A. G. E. (2019). Ten simple rules for the computational modeling of behavioral data. *eLife*, *8*, e49547. https://doi.org/10.7554/eLife.49547

Yellott, Jr., John I. (1977). The relationship between Luce's choice axiom, Thurstone's theory of comparative judgment, and the double exponential distribution. *Journal of Mathematical Psychology*, *15*(2), 109–144. https://doi.org/10.1016/0022-2496(77)90026-8

Yonelinas, A. P., & Jacoby, L. L. (2012). The process-dissociation approach two decades later: Convergence, boundary conditions, and new directions. *Memory & Cognition*, *40*(5), 663–680. https://doi.org/10.3758/s13421-012-0205-5

Appendix

Memory Updating: A More Complex Custom M^3

To illustrate how custom specifications can accommodate more complex task designs, we present a memory updating example based on Oberauer and Lewandowsky (2019). Unlike the filtering example in Tutorial 3 (which modifies only the distractor categories of the `cs` version), this example introduces entirely new response categories, nonlinear activation formulas with design variables, and three additional parameters beyond a and c.

In this task, participants memorize items at five positions, then each position is updated once. At test, participants identify the current target from a recognition set containing: the current target (1 option), other current items (4), the replaced item from the tested position (1), replaced items from other positions (4), and not-presented lures (5). This produces five response categories with the following activation formulas:

$$
\begin{aligned}
A_{\text{correct}} &= b + (1 + e \cdot t_e) \cdot (a + c) \\
A_{\text{other}} &= b + (1 + e \cdot t_e) \cdot a \\
A_{\text{oldinpos}} &= b + \exp(-r \cdot t_r) \cdot d \cdot (1 + e \cdot t_e) \cdot (a + c) \\
A_{\text{otherold}} &= b + \exp(-r \cdot t_r) \cdot d \cdot (1 + e \cdot t_e) \cdot a \\
A_{\text{NPL}} &= b
\end{aligned}
$$

Three new parameters appear: d captures the residual activation of replaced items (logit link, bounded 0–1), e scales the benefit of extended encoding time t_e (log link, positive), and r governs how quickly old-item activation decays with removal time t_r (log link, positive). The terms t_e and t_r are experimentally manipulated design variables (each at 0.25, 0.5, and 1.0 s), producing a 3 × 3 within-subjects design with 9 conditions.

The model definition:

```
m3_model_updating <- m3(
  resp_cats = c("correct", "other", "oldinpos", "otherold", "npl"),
```

```
  num_options = c(
    "n_correct" = 1, "n_other" = 4, "n_oldinpos" = 1,
    "n_otherold" = 4, "n_npl" = 5
  ),
  version     = "custom",
  choice_rule = "softmax",
  links = list(
    a = "identity", c = "identity",
    d = "logit", e = "log", r = "log"
  )
)

m3_act_funs_updating <- bmf(
  correct  ~ (1 + e * te) * (a + c) + b,
  other    ~ (1 + e * te) * a + b,
  oldinpos ~ exp(-r * tr) * d * (1 + e * te) * (a + c) + b,
  otherold ~ exp(-r * tr) * d * (1 + e * te) * a + b,
  npl      ~ b
)
```

Note that *te* and *tr* appear in the formulas but are not estimated parameters — they are data columns whose values vary across conditions. When *bmm* encounters variables in activation formulas that are not listed as model parameters, it treats them as data columns. This allows custom specifications to incorporate experimentally manipulated design variables directly into the activation equations.

Complete parameter recovery simulations for this model — varying sample size ($N \in \{25,50,100\}$) and trials per condition ($\{5,10,25\}$) — are available in the companion scripts (*scripts/tutorial5_parameter_recovery_updating_simple.R* for a single cell walkthrough, *scripts/tutorial5_parameter_recovery_updating.R* for the full design grid) and in the online supplement. These scripts demonstrate how recovery quality depends on both the number of participants and the amount of data per participant, and can serve as templates for validating other custom specifications.